DispatchX

Стійка система управління логістикою та складськими запасами

Системна документація — v1.0

2026

1. Вступ

DispatchX — це децентралізована, offline-first веб-платформа для управління логістичними ланцюгами та складськими запасами в умовах обмеженого постачання, нестабільного зв'язку та кризових операцій.

Головна архітектурна мета — оптимізація розподілу обмежених ресурсів за допомогою детермінованої математичної моделі — Smart Triage Engine — замість класичної черги FIFO.

Система побудована як Progressive Web Application (PWA), що забезпечує роботу на пристроях від диспетчерських робочих станцій до мобільних терміналів водіїв у зонах з нестабільним інтернет-з'єднанням.

1.1 Призначення документа

Цей документ містить повну технічну та користувацьку документацію системи DispatchX. Охоплює:

- Архітектуру системи та технологічні рішення
- Детальний опис алгоритму Smart Triage Engine
- Рольовий доступ і робочі процеси користувачів
- Інструкції з налаштування та розгортання для розробників
- Довідник API та моделей даних

1.2 Цільова аудиторія

Аудиторія	Відношення до документа
Frontend / Backend розробники	Архітектура, структура кодової бази, реалізація алгоритмів, розгортання
Диспетчери (логісти)	Робота з дашбордом, управління маніфестами, глобальний радар, контроль флоту
Водії (експедитори)	Мобільний HUD, навігація маніфестом, SOS-протокол
Представники хабів	Оформлення замовлень, каталог SKU, таймер відсічення, статус замовлення

2. Архітектура системи

2.1 Технологічний стек

Рівень	Технологія	Примітки
Фреймворк	React 18+ (Vite)	HMR, оптимізований білд-пайплайн
Стейт-менеджмент	React Context API	GlobalStore.jsx, AuthContext.jsx
Маршрутизація	React Router	ProtectedRoute.jsx для кожної ролі
ГІС / Карти	Leaflet + React-Leaflet	Тайли OpenStreetMap
Офлайн / PWA	Service Workers + Workbox	Стратегія Stale-While-Revalidate
Встановлення PWA	Хук UsePWAInstall.js	Windows / Android / iOS

2.2 Offline-First парадигма

DispatchX розроблений для роботи без підключення до мережі. Офлайн-архітектура складається з двох рівнів:

Service Workers та Workbox

Оболонка застосунку (HTML, CSS, JS-бандли) кешується при першому завантаженні. Наступні API-запити використовують стратегію Stale-While-Revalidate: кешована відповідь повертається негайно, поки у фоні відбувається ревалідація та оновлення кешу.

Кастомні React-хуки

- UseOffline.js — глобальний лісенер стану мережі (події online/offline). Тригерить UI-попередження та блокує всі мутації даних, що залежать від сервера. Ставить мутації в чергу для відтворення після відновлення з'єднання.
- UsePWAInstall.js — управляє життєвим циклом beforeinstallprompt, що дозволяє встановлювати систему як нативний застосунок на Windows, Android та iOS.

2.3 Структура проєкту

```
├── public/                    # Статичні ресурси, маніфест PWA
│   ├── favicon.svg
│   ├── icon-192.png
│   ├── icon-512.png
│   └── icons.svg
├── dev-dist/                  # Вивід SW для розробки (Workbox)
│   ├── sw.js
│   └── workbox-2ae722a1.js
├── src/
│   ├── assets/                # Графіка та ілюстрації
│   ├── components/            # Базові UI-компоненти
```

```
│   │   ├── features/
│   │   │   ├── auth/
│   │   │   │   └── LoginForm.jsx
│   │   │   ├── dashboard/
│   │   │   │   ├── AddRequestForm.jsx
│   │   │   │   ├── ClientDashboard.jsx
│   │   │   │   ├── Dashboard.jsx
│   │   │   │   ├── DriverDashboard.jsx
│   │   │   │   ├── RequestCard.jsx
│   │   │   │   └── StatsGrid.jsx
│   │   │   └── map/
│   │   │       └── MapViewer.jsx
│   │   ├── Button.jsx
│   │   ├── DeliveryCard.jsx
│   │   ├── ProtectedRoute.jsx
│   │   └── StatusBadge.jsx
│   ├── constants/              # Глобальні константи
│   │   ├── routes.js
│   │   └── statuses.js
│   ├── context/                # Провайдери глобального стану
│   │   ├── AuthContext.jsx
│   │   ├── GlobalStore.jsx
│   │   ├── LanguageContext.jsx
│   │   └── NotificationContext.jsx
│   ├── data/                   # Моки даних (розробка/тестування)
│   │   └── mockData.js
│   ├── hooks/                  # Кастомні React-хуки
│   │   ├── UseOffline.js
│   │   └── UsePWAInstall.js
│   ├── layouts/                # Обгортки сторінок
│   │   ├── AuthLayout.jsx
│   │   └── MainLayout.jsx
│   ├── pages/                  # Компоненти сторінок (рівень маршрутів)
│   │   ├── AnalyticsPage.jsx
│   │   ├── DashboardPage.jsx
│   │   ├── InventoryPage.jsx
│   │   ├── LoginPage.jsx
│   │   ├── MapPage.jsx
│   │   └── SettingsPage.jsx
│   ├── utils/                  # Чисті JS-алгоритми
│   │   ├── generateIcons.js
│   │   ├── priority.js         # Smart Triage Engine
│   │   └── validators.js
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── index.html                  # Точка входу
```

```
├── vite.config.js
├── eslint.config.js
└── package.json
```

3. Smart Triage Engine

Smart Triage Engine — це основний алгоритмічний модуль DispatchX, реалізований у src/utils/priority.js. Він генерує агрегований TotalScore для кожної логістичної транзакції. Заявки з найвищим балом автоматично піднімаються в топ черги диспетчера.

Рушій запускається при кожній ітерації: при створенні заявки, при зміні залишків та за розкладом.

3.1 Формула розрахунку

Формула: TotalScore = BaseScore + WaitTimeBonus + BurnRateOverride + DeferralBonus + SpatialBonus

3.2 Деталізація компонентів

BaseScore — Базовий скоринг за статусом

Ініціалізує пріоритет на основі задекларованої терміновості заявки. Включає систему квотування (Quota Management) для запобігання зловживанням (наприклад, коли користувач безпідставно маркує всі заявки як критичні).

Статус	Бали	Умова
Нормальний	+10	Завжди
Підвищений	+50	Завжди
Критичний	**+100**	Лише якщо quotaExceeded === false

WaitTimeBonus — Запобігання голодуванню

Лінійна часова функція, яка гарантує, що низькопріоритетні заявки з часом будуть виконані. Унеможливлює нескінченне очікування в черзі.

Формула: +2 бали за кожну повну годину в черзі з моменту request.timestamp (Date.now() - request.timestamp)

BurnRateOverride — Проактивний контроль виснаження

Найважливіший тригер системи. Аналізує відношення поточного залишку товару (currentStock) до швидкості його споживання (burnRate, одиниць/год). Якщо час до повного виснаження менше 2 годин або абсолютний залишок менше 20 одиниць — алгоритм примусово ігнорує базові статуси.

Умова тригера: Time-to-Empty < 2год АБО currentStock < 20 одиниць → примусово +200 балів

DeferralBonus — Кооперативний стимул (теорія ігор)

Якщо вузол раніше добровільно поступився своєю чергою (hasDeferred: true), система запам'ятовує це та нараховує компенсаційні бали при наступній ітерації. Стимулює кооперативну поведінку між конкуруючими вузлами постачання.

Умова: hasDeferred === true → +30 балів при наступній ітерації

SpatialBonus — Просторова оптимізація маршруту

При формуванні маніфесту транспортного засобу рушій аналізує топологію маршруту. Якщо заявка лежить на векторі руху вже сформованої фури (routeMatch === true), вона отримує бонус для максимізації коефіцієнта завантаженості флоту.

Умова: routeMatch === true → +80 балів

4. Рольова модель доступу (RBAC)

MainLayout.jsx динамічно монтує різні робочі середовища залежно від автентифікованої ролі користувача з AuthContext. Система має три окремі ролі, кожна з налаштованим UI та набором прав доступу.

4.1 Диспетчер (Системний адміністратор / Логіст)

Робоче середовище: High-Density дашборд "Командний пункт" — повний контроль системи.

Глобальний радар

- Інтерактивна карта Leaflet з відображенням цифрових двійників усіх транспортних засобів флоту в реальному часі.
- Координати транспорту перераховуються кожні 200 мс по полілініях для точного розрахунку ETA.
- Маркери з кольоровим кодуванням відображають завантаженість та час прибуття.

Управління флотом

- Конструктор маніфестів: призначення заявок транспортним засобам.
- Панель вантажності (Capacity Bar): валідація максимального навантаження в реальному часі. Запобігає перевантаженню.
- Блокування маніфесту: після затвердження маніфест стає незмінним для представників хабів.

Глобальний склад

- Синхронізована таблиця обліку SKU по всіх хабах (наприклад, Lviv West Hub, Odesa Port Hub).
- Залишки на складі, burn rate та статус наявності для кожного хабу в єдиному вікні.
- Вбудоване редагування коригувань залишків та ручне перевизначення пріоритетів.

Розумна черга

- Черга пріоритетів у реальному часі, відсортована за TotalScore (найвищий — перший).
- Вбудоване відображення розбивки балів для кожної заявки (BaseScore, BurnRateOverride тощо).
- Затвердження заявки та призначення до маніфесту в один клік.

4.2 Водій (Експедитор)

Робоче середовище: Mobile/Tablet-first HUD — мінімальне когнітивне навантаження, оптимізовано для однієї руки.

Навігація по активному маніфесту

- Список призначених точок доставки, відсортованих за оптимізованим маршрутом.
- Велика кнопка підтвердження для кожної точки: доставлено / частково / проблема.
- Автоматичний перехід до наступної точки після підтвердження.

SOS-кризовий протокол

- Спеціальна кнопка SOS асинхронно надсилає подію issue_reported_toast в UI диспетчера.
- Диспетчер отримує миттєве сповіщення з ID транспорту, локацією та типом проблеми.
- Дозволяє негайно перенаправити резервний транспорт на перехоплення вантажу.
- Всі SOS-події логуються з часовою міткою та GPS-координатами.

Офлайн-режим

- Повний маніфест доступний офлайн — завантажується в кеш при затвердженні маніфесту.
- Підтвердження доставок ставляться в чергу локально та синхронізуються при відновленні з'єднання.

4.3 Представник хабу (Клієнт / Отримувач)

Робоче середовище: B2B e-Commerce портал — оформлення замовлень та відстеження статусу.

Каталог SKU

- Перегляд доступних номенклатурних позицій (SKU) з поточними залишками на складі.
- Для кожного SKU відображається найближчий складський хаб.
- Фільтрація за категорією, наявністю та рівнем терміновості.

Оформлення заявки

- Створення заявок зі статусом: Нормальний, Підвищений або Критичний (з контролем квот).
- Вказання кількості, бажаного вікна доставки та хабу-отримувача.
- Заявка надходить до черги пріоритетів диспетчера з початковим TotalScore.

Таймер відсічення (Cut-off Timer)

- Кожна заявка відображає зворотній відлік до наступного вікна відправки.
- Після затвердження диспетчером маніфесту із заявкою — статус переходить у Locked (Заблоковано).
- Заблоковані заявки не можуть бути змінені представником хабу — запобігає пошкодженню маніфесту.

Відстеження замовлення

- Оновлення статусу в реальному часі: Очікує → В черзі → Призначено → В дорозі → Доставлено.
- ETA відображається на основі GPS-позиції транспорту та розрахунку маршруту.

5. Гайд розробника

5.1 Вимоги

Вимога	Версія	Примітки
Node.js	18+	Рекомендується LTS
npm	9+	Входить до складу Node.js
Git	Будь-яка актуальна	Для клонування репозиторію

5.2 Встановлення та запуск

Клонування репозиторію та встановлення залежностей:

```
git clone <repository_url>
cd dispatchx
npm install
```

Запуск сервера розробки з Hot Module Replacement:

```
npm run dev
```

Збірка для продакшену (також генерує Service Workers через Workbox):

```
npm run build
```

Тестування продакшн-збірки локально:

```
npm run preview
```

5.3 Змінні середовища

Створіть файл .env у кореневій директорії проєкту. Підтримуються наступні змінні:

```
VITE_API_BASE_URL=https://api.dispatchx.internal
VITE_MAP_TILE_URL=https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png
VITE_VEHICLE_UPDATE_INTERVAL_MS=200
VITE_BURNRATE_CRITICAL_HOURS=2
VITE_BURNRATE_CRITICAL_STOCK=20
```

5.4 Ключові конфігураційні файли

Файл	Призначення
src/constants/statuses.js	Визначає всі рядки статусів заявок та допустимі переходи між ними
src/constants/routes.js	Всі клієнтські шляхи маршрутизації, згруповані за роллю

src/data/mockData.js	Сідові дані для локальної розробки та офлайн-тестування
src/utils/priority.js	Smart Triage Engine — основний алгоритм пріоритезації
src/utils/validators.js	Хелпери валідації введення для форм та API-пейлоадів
src/utils/generateIcons.js	Утиліта для динамічної генерації SVG-іконок
src/context/GlobalStore.jsx	Глобальний стан: заявки, запаси, транспорт, маніфести
src/context/AuthContext.jsx	Стан автентифікації та управління ролями
src/context/LanguageContext.jsx	Стан перемикання мови інтерфейсу (i18n)
src/context/NotificationContext.jsx	Стан та диспетчер toast/alert-сповіщень

6. Основні моделі даних

6.1 Об'єкт заявки (Request)

```
{
  id:            string,      // Унікальний ID заявки
  status:        string,      // 'normal' | 'elevated' | 'critical'
  timestamp:     number,      // Unix мс — час створення (для WaitTimeBonus)
  hubId:         string,      // Ідентифікатор хабу-призначення
  skuId:         string,      // Посилання на SKU товару
  quantity:      number,      // Запитувана кількість одиниць
  hasDeferred:   boolean,     // true якщо вузол добровільно поступився чергою
  quotaExceeded: boolean,     // true якщо перевищено квоту критичних для вузла
  routeMatch:    boolean,     // true якщо заявка на векторі активного маршруту
  lockedAt:      number|null, // Unix мс — час затвердження маніфесту
  totalScore:    number,      // Розраховується priority.js
}
```

6.2 Об'єкт запасів / SKU

```
{
  skuId:        string,   // Унікальний ідентифікатор товару
  name:         string,   // Назва товару для відображення
  hubId:        string,   // Хаб зберігання
  currentStock: number,   // Поточний залишок на складі
  burnRate:     number,   // Одиниць споживання на годину (ковзне середнє)
  unit:         string,   // 'кг' | 'шт' | 'л' тощо
}
```

6.3 Об'єкт транспортного засобу

```
{
  id:            string,            // Ідентифікатор транспорту
  driverId:      string,            // ID призначеного водія
  status:        string,            // 'idle' | 'en_route' | 'delivering'
  coordinates:   [number, number],  // [lat, lng] — оновлюється кожні 200мс
  routePolyline: [number,number][], // Впорядковані точки маршруту
  manifestId:    string|null,       // Посилання на активний маніфест
  maxCapacity:   number,            // Максимальне навантаження в кг
  currentLoad:   number,            // Поточне навантаження в кг
}
```

7. Події системи

DispatchX використовує легковаговану внутрішню шину подій для міжмодульної комунікації. Ключові події:

Подія	Джерело	Ефект
issue_reported_toast	HUD водія	Викликає SOS-сповіщення в інтерфейсі диспетчера
manifest_locked	Диспетчер	Встановлює request.lockedAt; блокує мутації на стороні хабу
stock_updated	Будь-яка роль	Тригерить перерахунок priority.js для відповідних SKU
vehicle_position_update	Симуляція / GPS	Оновлює маркери на карті та розрахунки ETA
offline_detected	Хук UseOffline	Вимикає кнопки мутацій; показує офлайн-банер
online_restored	Хук UseOffline	Відтворює чергу мутацій; синхронізується з сервером

8. Глосарій

Термін	Визначення
BurnRate	Швидкість споживання запасів в одиницях на годину. Використовується для розрахунку Time-to-Empty.
TotalScore	Агрегований бал пріоритету, розрахований Smart Triage Engine для кожної заявки.
DeferralBonus	Компенсаційний бонус для вузлів, що раніше добровільно поступилися своїм місцем у черзі.
SpatialBonus	Бонус балів для заявок, що географічно знаходяться на маршруті вже відправленого транспорту.
BurnRateOverride	Аварійний буст балів (+200), що тригериться при загрозі виснаження запасів.
WaitTimeBonus	Лінійний за часом антиголодуючий бонус для заявок, що очікують у черзі.
Маніфест	Затверджене, незмінне призначення заявок конкретному транспортному засобу для доставки.
Cut-off Timer	Зворотній відлік до наступного вікна відправки; заморожує замовлення після блокування маніфесту.
Digital Twin	Віртуальне представлення фізичного транспортного засобу на карті Глобального радара в реальному часі.
SKU	Stock Keeping Unit — унікальний ідентифікатор типу товару на складі.
PWA	Progressive Web Application — веб-застосунок, що встановлюється та працює як нативний, з підтримкою офлайн.
SOS-протокол	Ініційована водієм аварійна подія, що сповіщає диспетчера для негайного перенаправлення транспорту.
Quota Management	Системне обмеження кількості критичних заявок на вузол для запобігання зловживанням.
routeMatch	Булевий прапор, що вказує на збіг пункту призначення заявки з вектором руху активного маршруту.

— Кінець документа —